# How Curriculums Affect the Training Dynamics of Small LMs?

*Training Dynamics of a Small GPT2 on WikiText-103*

Yehong Qiu

## Abstract

In this work, we investigate the effect of curriculum learning on the training dynamics of a small transformer language model trained on WikiText-103. We compare curriculum, anti-curriculum, and random data ordering under two different difficulty definitions: unigram and bigram surprisal scores. In addition to standard train and test loss, we track internal dynamics using spectral divergence and representation anisotropy.

We find that curriculum learning consistently accelerates optimization and shifts an apparent internal critical point earlier in training. However, its effect on final test performance is modest and highly sensitive to the choice of difficulty metric. Notably, while performance gains disappear under bigram-based difficulty, the qualitative patterns in internal dynamics remain largely invariant.

These observations suggest that curriculum learning may primarily act as a mechanism that alters the training trajectory rather than consistently improving final performance. We discuss the implications of this perspective and raise open questions about whether such trajectory-level effects persist under scaling and how they may be leveraged in large-scale language model training.

## 1 Introduction

The order in which training data is presented to a model can significantly influence the learning process. Curriculum learning, which organizes data from easy to difficult, has been proposed as a way to improve optimization and generalization[1]. While it has shown empirical benefits in various domains, its role in language model training remains ambiguous[2].

Most existing studies evaluate curriculum learning primarily through its effect on final performance metrics such as test loss. However, this perspective may overlook an important aspect of the learning process: the trajectory through which a model reaches its final state. In highly non-convex optimization landscapes, different training paths may lead to similar performance but differ substantially in terms of training efficiency, stability, and internal representations.

In this work, we study curriculum learning from a training dynamics perspective. We conduct controlled experiments on a small transformer language model trained on WikiText-103, comparing curriculum, anti-curriculum, and random data ordering strategies. We further introduce two distinct definitions of data difficulty—unigram and bigram surprisal—to examine how the notion of "difficulty" interacts with training behavior.

Our goal is not only to evaluate whether curriculum learning improves performance, but also to understand what aspect of training it influences. Specifically, we investigate the following questions:

(1) Does curriculum learning consistently improve final model performance, or are its effects dependent on how difficulty is defined?

(2) Does curriculum learning modify the optimization trajectory, and if so, how is this reflected in internal model dynamics?

(3) Are the effects of curriculum learning primarily performance-driven, or do they manifest more strongly in training efficiency and internal representation structure?

To address these questions, we analyze both external metrics (train/test loss) and internal probing signals (spectral divergence and anisotropy). Our results show that while performance gains are inconsistent, curriculum learning systematically alters the training trajectory, suggesting that its primary role may lie in shaping the optimization process rather than directly improving final outcomes.

These findings motivate a shift in perspective: instead of viewing curriculum learning solely as a tool for boosting performance, it may be more appropriately understood as a mechanism for controlling training dynamics. This raises further questions about whether such effects persist at scale and how they can be exploited in large-scale language model training.

# 2 Experiment Setup

The experiements are conducted on a 2-layer transformer model fed with natural language from Wikipedia, and curriculums are designed as the order in which the training data is fed into the model. Data is fed into the model training pipeline either from easy to hard(curriculum), from hard to easy(anti-curriculum), or randomly(baseline). The train and test loss are tracked throughout the process of training, as well as other auxiliary probing metrics.

## 2.1 Dataset and Data Preprocessing

The dataset used in the experiments is the open-sourced WikiText-103, containing high quality texts from Wikipedia. A train set containing 6.5M tokens is extracted from the original dataset and tokenized with a pre-trained GPT2 tokenizer. A smaller disjoint set tokenized with the same tokenizer is used as the test set.

Both the train and the test set are segmented into equal-length sequences with a stride being half the length of each sequence, ensuring natural information overlapping across sequences. Padding is applied to those sequences whose actual lengths are smaller than the designated length, and padded tokens are left out in the loss computation during training and testing.

The data processing protocol is shown in the flow chart 1 below. The difficulty score computation and data segmentation will be introduced soon in later sections.

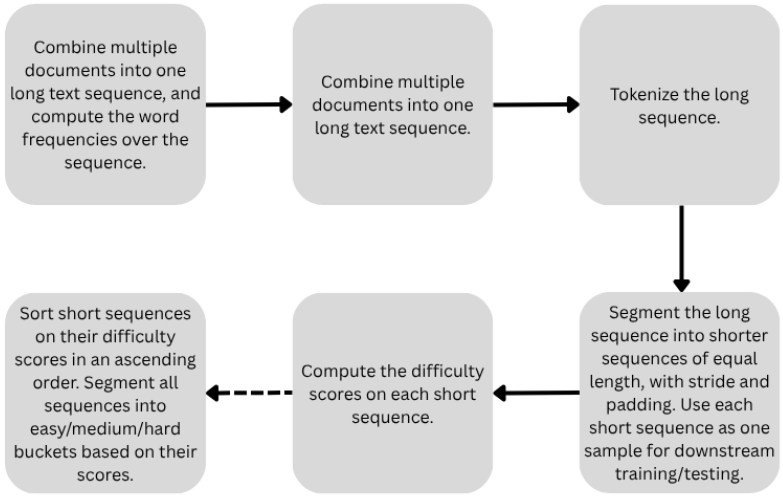


Figure 1: The data processing protocol

## 2.2 Model Structure and Training Scheme

A small 2-layer decoder-transformer is used to make the training dynamics observable and computationally tractable within a controlled experiment setting. The model architecture follows a pre-LayerNorm GPT2 model with causal masked self-attention.[3] The model hyperparameters are listed in table 1.

| Layers | Context Length | Embedding Dim | Heads | FFN Dim |
|---|---|---|---|---|
| 2 | 128 | 256 | 4 | 1024 |

Table 1: Model Architecture Parameters

The baseline experiment is to feed the data randomly into the model during each epoch. It is used as a control group in the study of curriculum learning.

The training hyperparameters for the baseline experiment are listed in table 2.

| Name | Configurations |
|---|---|
| Batch Size | 128 |
| Epochs | 100 |
| Optimizer | AdamW |
| Base Lr | 3e-4 |
| Betas | (0.9, 0.95) |
| Lr Scheduler | First 10% steps Warmup + CosAnneal to 10% Base Lr |
| Weight Decay | 0.1 |
| Grad Clip | 1.0 |

Table 2: Training Hyperparameters

The training loss is the average cross entropy loss over each batch:

$$l_\theta(\{x_t\}) \doteq \hat{E}_{x,t}(-log(\hat{p}(x_t|x_{<t}))$$

The test loss is also the negative log-likelihood averaged over every token in the test sample.

## 2.3 Curriculum Designs

There are two parts in the curriculum design: the difficulty definition and the pacing schema. The former determines how difficult each case is, and the latter determines in what pacing the model sees the case from easy to difficult.

### 2.3.1 Difficulty Score

The difficulty level of each text sequence is defined based on the usage frequency of each word within the sequence. The word frequency is computed merely on the training set to avoid data leakage. Note that for each tokenized sequence, it is first de-tokenized and the score is computed at a word level.

Specifically, there are two different scores used in the experiments, the unigram surprisal score and the bigram surprisal score. The unigram surprisal score evaluates the rarity of each individual word, while the bigram surprisal score measures the rarity of word transitions. The larger the scores are, the less likely each word/word tuple appears, thus indicating the harder the text sequences are.

The unigram surprisal score is formulated as:

$$\text{UnigramSS(X)} = \hat{E}_t(-log(\hat{p}(x_t))$$

The bigram surprisal score is:

$$\text{BigramSS(X)} = \hat{E}_t(-log(\hat{p}(x_t|x_{t-1}))$$

The two surprisal scores stress on different aspects of difficulty, and controlled experiments on the two score metrics can potentially illustrate the invariants in the training dynamics of curriculum learning.

### 2.3.2 Pacing Schema

The pacing schema for the curriculum learning used in the experiments consists of 4 phases:

$$\text{Easy} \rightarrow \text{Medium} \rightarrow \text{Hard} \rightarrow \text{Mixed}$$

The train set is dissected roughly into three buckets: easy, medium, and hard. The easy case bucket contains the text sequences whose scores are within the top $\sim 33.3\%$ quantile, the medium is in the $33.3\% \sim 66.7\%$ quantile, and the hard is within the quantile $> 66.7\%$.

During the first 3 phases of the curriculum learning, the model is trained on each bucket with an assigned 90 epochs, and then trained on all data with a designated 10 epochs, but the early stopping mechanism is introduced during every phase to prevent overfitting, and the leftover epochs from the previous 3 phases are recorded to amoritize into extra epochs during the last training phase on all the data. The epoch designation rationale is to provide an equal amount of data exposure as the baseline training: Since each difficulty bucket only contains roughly 1/3 of all the data, they should be trained on $30 \times 3$ epochs. But if the model is overfitted when trained on a single bucket, the leftover epochs should be first amoritized (divided by 3), and then added to the 10 epochs assigned to the last phase.

At the start of every phase, the learning rate scheduler is re-initiated. But in each phase, the learning rate is scheduled in the same way as that in the baseline training.

### 2.3.3 Anti-curriculum

An anti-curriculum learning schema is introduced to provide an extra control[1]. Similar to the curriculum learning schema, the anti-curriculum schema contains 4 phases, but the first 3 phases are in a reversed order of the curriculum schema.

## 2.4 Training Dynamics Probing

Apart from the train loss and the test loss score, two auxiliary probing metrics are computed throughout training: they are Spectral Divergence and Anisotropy. Tracking these metrics provides a glimpse into the model's internal phase during training.[4]

### 2.4.1 Spectral Divergence

Denote $\{\sigma_i\}_{i=1}^{r}$ as the singular values of the language model head matrix $W$, then normalize the squares of singular values to form a distribution $p_i = \sigma_i^2 / \Sigma_j \sigma_j^2$. Spectral Divergence is defined as the KL divergence between this distribution and the uniform distribution:

$$H_{sd}(W) = D_{KL}(p||U) = \Sigma_{i=1}^{r} p_i \log(r \cdot p_i)$$

So, the larger the metric is, the larger the spectral distribution from the uniform distribution, indicating that the singular values collapses into a couple of dominant values.

#### 2.4.2 Anisotropy

For a model checkpoint, the contextual representation is derived by sending an input sequence through the model and giving out the representation vector before it is forwarded into the model's last layer head matrix.

Anisotropy is defined on the contextual representations $H$ over a set of n sequences:

$$\mathcal{A}(H) = \frac{1}{n^2 - n} \Sigma_{h_i, h_j \in H, i \neq j} \frac{h_i^T h_j}{||h_i||_2 ||h_j||_2}$$

In which $h_i$ denotes the $i$th row of $H$. Thus, anisotropy is a normalized cosine-similarity over the n row vectors representing each of the input text sequences. The metric is interpreted as how the model maps different inputs to its internal representation space; the smaller the value is, the more diversed the direction each input's representation vector points to in the internal space.

# 3 Experiment Results and Analyses

## 3.1 Main Results

**UnigramSS-based curriculum outperforms consistently but modestly**

After bucketing the hold-out validation data into the easy, medium, and hard groups, the model test loss is computed on each difficulty bucket as well as on all of the hold-out data. In the table 3, the test losses of the model trained in the unigram curriculum learning outperforms the baseline(random data ordering), as well as the anti-curriculum learning.

| | Easy | Medium | Hard | Overall |
|---|---|---|---|---|
| Random | 4.4363 | 4.7064 | 5.0562 | 4.7329 |
| Curriculum | **4.4245** | **4.6780** | **4.9808** | **4.6944** |
| Anti-curriculum | 4.4777 | 4.7306 | 5.0427 | 4.7503 |

Table 3: Test losses with respect to each difficulty bucket as well as over all the hold-out data of the baseline training, curriculum, and anti-curriculum learning on the unigram surprisal score

This result indicates that curriculum learning has the potential to act as an efficient regularizer in searching for better minima.

### Curriculum speeds up the optimization process

As shown in fig 2, there are non-smooth jumps in each phase-transition point during the curriculum and anti-curriculum learning. After each of these jumps, the training loss curve of curriculum learning is steeper than baseline, indicating that the exposure to a curriculum can speed up optimization.

The test loss demonstrates a similar pattern but the reduction of test loss in the curriculum learning is monotonous.

The anti-curriculum learning shows a sharp surge in loss both in-sample and out-of-sample, when the learning pace shifts from medium to easy. This indicates the inefficiency of anti-curriculum learning, as the loss first goes down in a way not as efficient as the baseline and the curriculum learning during the early phases, and the loss goes up to a high value in the later phase, making the model wastes unnecessary time during training.

An interesting result here is that the curriculum learning can achieve similar or even better out-of-sample performance with much less exposure to the training data, and a relatively more exposure to the easy samples than to harder samples. As per the design of the pacing schema, each training phase will stop when the model overfits on that set of data used for the current phase, making the learning process finish faster while also reduce its exposure times to the training data. Besides, though the pacing is originally designed to provide equal exposure to every sample in the training set, the introduction of the early-stopping mechanism and the actual training dynamics end up making the model expose to the easy bucket in more times, and less to the medium and hard buckets.

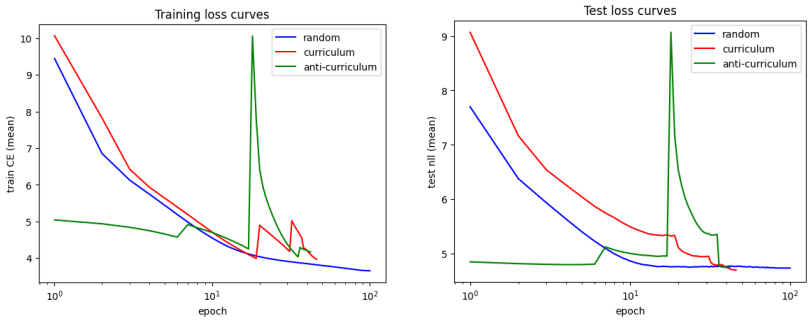


Figure 2: The train and test loss curves of the random baseline, the curriculum and anti-curriculum learning based on the unigram surprisal score

### Curriculum brings a critical point in advance

An interesting phenomenon found in the experiment is that both the baseline and the curriculum learning show a similar pattern on spectral divergence and anisotropy: the spectral divergence stays small at the start and quickly goes up in the later stage of training; the anisotropy goes through a hill before coming

down again. The increase in the value of spectral divergence and the surging of the hill-like structure in anisotropy is roughly simultaneous, and this seems to indicate a critical internal phase-changing point in the process of training. The only difference is that curriculum learning brings the critical point in advance.

The anti-curriculum learning, on the other hand, has completely different changing patterns in the two probing metrics. All these results open questions about the mechanism behind the learning process of the transformer architecture.

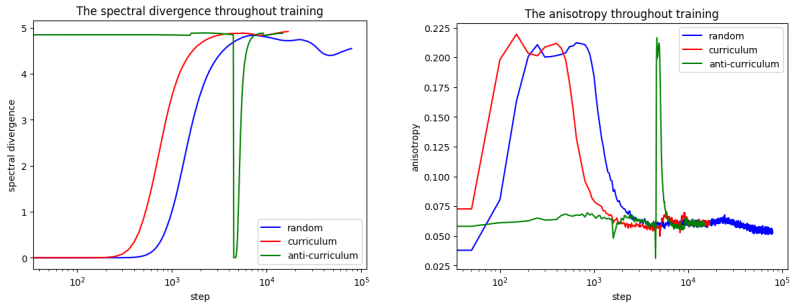


Figure 3: Spectral Divergence and Anisotropy of the random baseline, the curriculum, as well as the anti-curriculum learning based on the unigram surprisal score

# 4 Ablation Studies

To test the robustness of the above three main propositions, the unigram surprisal score is replaced by the bigram surprisal score, and the above experiments are executed again. It turns out that the modest performance gain of the curriculum learning on the test set disappears after the definition of difficulty changes, meaning that the efficiency of a curriculum is related to the design of the difficulty score.

| | Easy | Medium | Hard | Overall |
|---|---|---|---|---|
| Random | **4.4363** | **4.7064** | 5.0562 | **4.7329** |
| Curriculum | 4.4812 | 4.7350 | **5.0521** | 4.7560 |
| Anti-curriculum | 4.4925 | 4.7472 | 5.0680 | 4.7620 |

Table 4: Test losses with respect to each difficulty bucket as well as over all the hold-out data of the baseline training, curriculum, and anti-curriculum learning on the bigram surprisal score

On the other hand, the changing patterns of the probing metrics roughly stay the same, which is shown in fig 4. The interesting question here is what is the mechanism behind the invariance, which is a potential direction for future exploration.

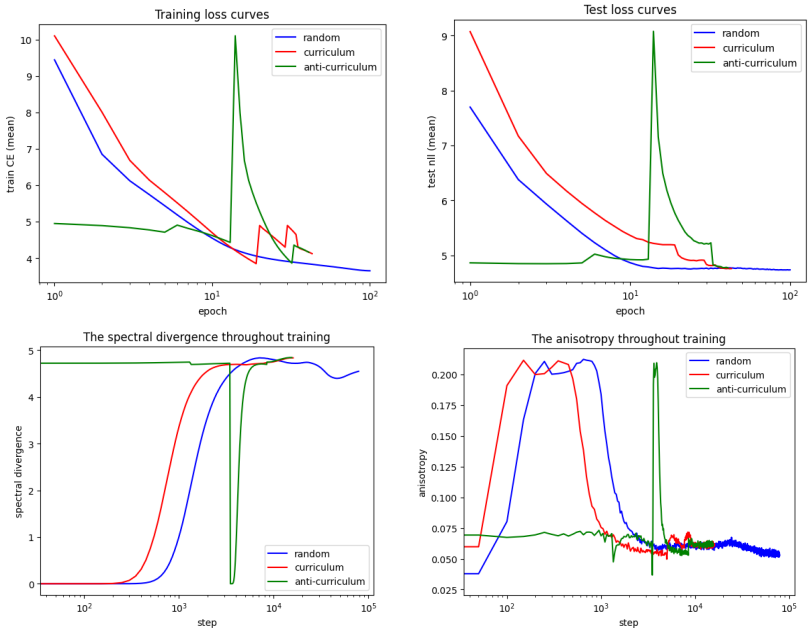


Figure 4: The training dynamics probing of the curriculum learning based on the bigram surprisal score

# 5 Discussion

The key take-away message from the series of experiments is that the definition of the difficulty score can change the eventual model performance, but something seems to be not changed across curriculum learning processes of different difficulty scores: the overall training speeds up, and the internal critical point is brought in advance by the curriculum learning.

If we propose that curriculum changes the training trajectory of LMs, a valuable question is whether this is true for larger transformer models. If this is true, introducing a well-designed curriculum learning into LLM training could lessen the time used for achieving similar performance, and thus saving the precious computation power and time.

But whether the road in front is fruitful remains obscure, one potential problem is that the speed-up effect in curriculum learning is due to the early-stopping introduced when over-fitting happens. LLMs are overparameterized, and whether they will overfit on any buckets of difficulty level remains unclear.

# References

[1] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," *Proceedings of the 26th International Conference on Machine Learning*

*(ICML)*, 2009.

[2] M. Elgaar and H. Amiri, "Curriculum learning for llm pretraining: An analysis of learning dynamics," *arXiv preprint arXiv:2601.21698*, 2026.

[3] GitHub repository: https://github.com/karpathy/minGPT.

[4] N. Godey, É. V. de la Clergerie, and B. Sagot, "Why do small language models underperform? studying language model saturation via the softmax bottleneck," in *Proceedings of the Conference on Language Modeling (COLM)*, 2024.